

GraphRAG 教学案例总览

本文档介绍本工程中的三个 GraphRAG 演示程序，帮助你理解不同实现方式的特点和适用场景。

演示程序一：内存版 GraphRAG

1.1 概述

文件：`local_graphrag_ollama.py`

特点：最简单的 GraphRAG 实现，使用 NetworkX 将知识图谱存储在内存中。适合快速原型开发和教学演示。

展示内容：

- GraphRAG 的核心工作流程
- 如何从文本中提取三元组
- 如何使用 NetworkX 构建知识图谱
- 如何进行本地（多跳）查询

1.2 运行方式

前置条件：

1. 安装依赖：`pip install -r requirements.txt`
2. 启动 Ollama 服务：`ollama serve`
3. 确保 qwen2.5 模型已拉取：`ollama pull qwen2.5`

运行命令：

```
python local_graphrag_ollama.py
```

启动脚本（可选）：

```
# 启动 Ollama（如果未运行）
ollama serve

# 运行程序
python local_graphrag_ollama.py
```

1.3 包含文件

文件	说明
<code>local_graphrag_ollama.py</code>	主程序文件

文件	说明
<code>requirements.txt</code>	Python 依赖配置

1.4 核心代码结构

```
class LocalGraphRAGStore:
    """使用 NetworkX 的内存图存储"""
    def __init__(self):
        self.graph = nx.DiGraph() # 有向图存储

    def add_triplet(self, subject, relation, obj):
        """添加三元组"""
        self.graph.add_node(subject)
        self.graph.add_node(obj)
        self.graph.add_edge(subject, obj, relation=relation)

    def get_graph(self):
        """获取图谱对象"""
        return self.graph
```

1.5 输出示例

```
=====
GraphRAG 本地知识库演示
=====

[初始化] 正在初始化 Ollama LLM (qwen2.5)...
[初始化] Ollama LLM 初始化完成

[提取] 正在提取知识三元组...
[提取] 提取完成, 共获得 7 个三元组
  1. 张三 -父亲-> 李四
  2. 李四 -工作单位-> 谷歌公司
  3. 谷歌公司 -位于-> 美国加州山景城
  ...

[图谱] 图谱构建完成! 节点: 8, 边: 7

=====
查询演示
=====

问题: 张三和谷歌公司有什么关系?
答案: 张三通过他的父亲李四与谷歌公司产生联系。
      路径: 张三 -> 父亲 -> 李四 -> 工作单位 -> 谷歌公司
```

1.6 适用场景

-  快速原型开发
-  教学演示
-  单一机器运行
-  不适合数据持久化
-  重启后数据丢失

演示程序二：Neo4j 版 GraphRAG

2.1 概述

文件： `local_graphrag_neo4j.py`

特点：使用 Neo4j 图数据库作为后端存储，实现数据的持久化。适合生产环境和复杂图查询。

展示内容：

- 如何将知识图谱存储到 Neo4j
- Neo4j 的 Cypher 查询语言
- 图数据库的持久化存储优势
- 复杂关系查询能力

2.2 运行方式

前置条件：

1. 安装依赖： `pip install -r requirements.txt`
2. 启动 Ollama 服务： `ollama serve`
3. 启动 Neo4j 服务： `neo4j start`
4. 确保 qwen2.5 模型已拉取： `ollama pull qwen2.5`
5. 配置 Neo4j 连接信息（在代码中或环境变量）

运行命令：

```
# 启动 Neo4j
neo4j start

# 运行程序
python local_graphrag_neo4j.py
```

启动/关闭脚本：

```
./start_graphrag.sh # 启动服务和运行程序
./stop_graphrag.sh  # 停止服务
```

2.3 包含文件

文件	说明
<code>local_graphrag_neo4j.py</code>	主程序文件
<code>start_graphrag.sh</code>	Neo4j 版本启动脚本
<code>stop_graphrag.sh</code>	Neo4j 版本停止脚本
<code>requirements.txt</code>	Python 依赖配置

2.4 核心代码结构

```
class Neo4jGraphRAGStore:
    """使用 Neo4j 的持久化图存储"""

    def __init__(self, uri="bolt://localhost:7687",
                 username="neo4j", password="reins2011!"):
        self.driver = GraphDatabase.driver(uri, auth=(username, password))

    def add_triplet(self, subject, relation, obj):
        """使用 Cypher 添加三元组"""
        with self.driver.session() as session:
            session.run("""
                MERGE (a:Entity {name: $subject})
                MERGE (b:Entity {name: $object})
                MERGE (a)-[r:`{relation}`]->(b)
            """, subject=subject, object=obj)

    def query(self, cypher):
        """执行 Cypher 查询"""
        with self.driver.session() as session:
            return session.run(cypher)
```

2.5 Neo4j Browser 可视化

运行后，可以在 Neo4j Browser (<http://localhost:7474>) 中查看知识图谱：

```
# 查看所有节点和关系
MATCH (a)-[r]->(b) RETURN a, r, b

# 查看图谱统计
MATCH (n) RETURN count(n) as nodeCount
MATCH ()-[r]->() RETURN count(r) as relationshipCount
```

2.6 输出示例

```
=====
GraphRAG + Neo4j 演示
```

```
=====

[连接] 正在连接 Neo4j 数据库...
[连接] Neo4j 连接成功 (bolt://localhost:7687)

[提取] 正在提取知识三元组...
[提取] 提取完成, 共获得 7 个三元组
    1. 张三 -父亲-> 李四
    2. 李四 -工作单位-> 谷歌公司
    ...

[存储] 正在存入 Neo4j...
[存储] 存储完成! 节点: 8, 边: 7

[验证] 验证数据...
    ✓ 节点数量: 8
    ✓ 边数量: 7

=====
```

查询演示

```
=====

问题: 张三和谷歌公司有什么关系?
答案: 张三通过他的父亲李四与谷歌公司产生联系。

=====
```

2.7 适用场景

- ✓ 数据持久化存储
- ✓ 复杂图查询
- ✓ 生产环境部署
- ✓ 多用户共享数据
- ✗ 需要额外安装 Neo4j

演示程序三：前后端对比系统

3.1 概述

文件位置：

- 后端：backend/app.py
- 前端：frontend/

特点：完整的 Web 应用，同时实现 GraphRAG 和普通 RAG，支持左右对比查询和知识图谱可视化。

展示内容：

- GraphRAG 与普通 RAG 的对比
- 两种系统的优劣势分析
- Web 界面交互
- 知识图谱可视化

- API 服务开发

3.2 运行方式

前置条件：

1. 安装 Python 依赖：`pip install -r requirements.txt`
2. 安装 Node.js 依赖：`cd frontend && npm install`
3. 启动 Ollama 服务：`ollama serve`
4. 启动 Neo4j 服务：`neo4j start`
5. 确保 qwen2.5 和 mxbai-embed-large 模型已拉取

运行命令：

```
# 方式一：使用完整启动脚本（推荐）
./start_full.sh

# 方式二：手动启动各服务
# 终端1：启动后端
python backend/app.py

# 终端2：启动前端
cd frontend
node server.js

# 终端3：确保 Neo4j 运行
neo4j start
```

访问地址：

- 前端页面：`http://localhost:3000`
- 后端 API：`http://localhost:5001`
- Neo4j Browser：`http://localhost:7474`

停止命令：

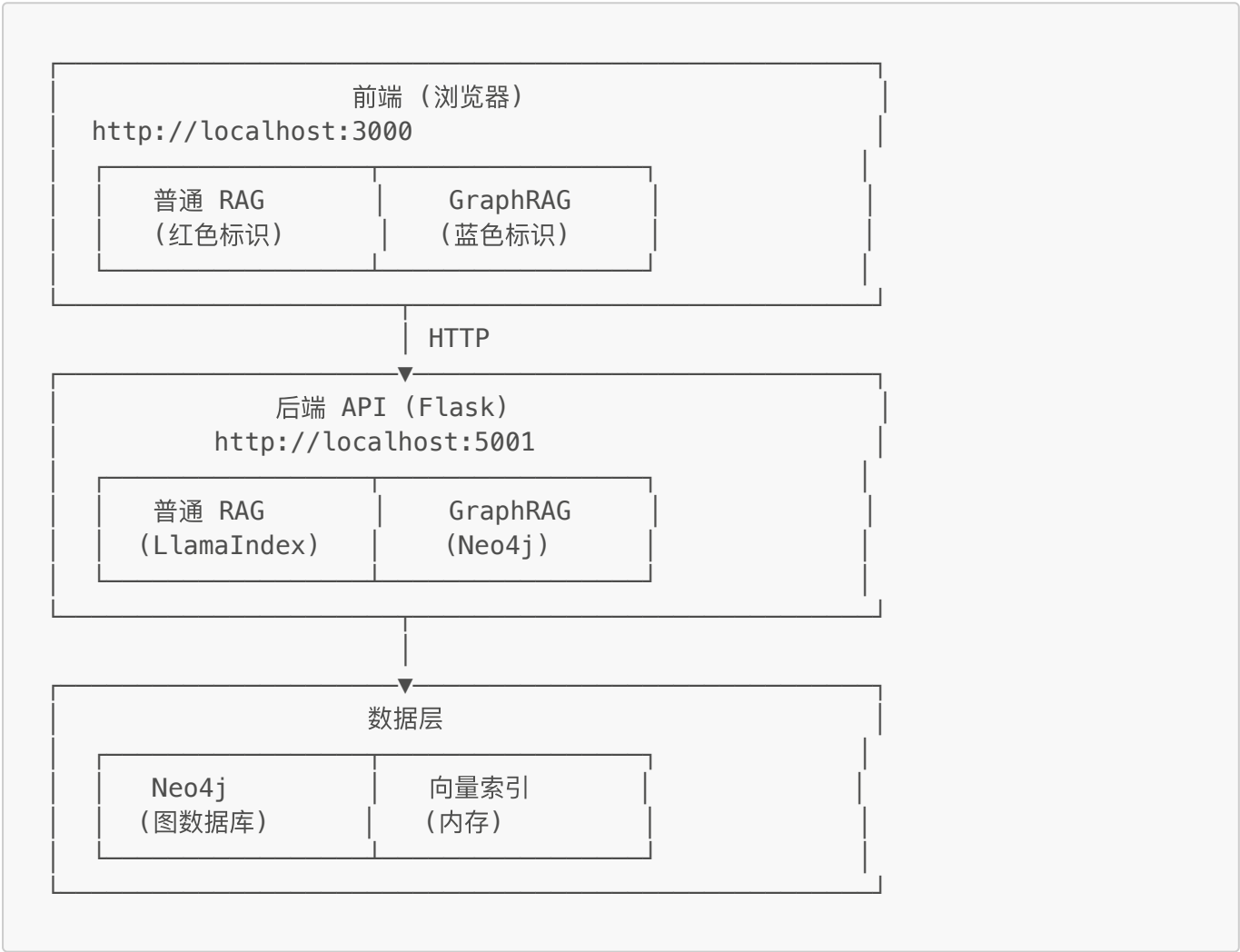
```
./stop_full.sh
```

3.3 包含文件

目录/文件	说明
backend/app.py	Flask 后端 API 服务
frontend/server.js	Node.js 前端代理服务
frontend/package.json	前端依赖配置
frontend/public/index.html	前端页面

目录/文件	说明
frontend/public/style.css	样式文件
frontend/public/app.js	前端逻辑
start_full.sh	完整启动脚本
stop_full.sh	完整停止脚本
requirements.txt	Python 依赖配置

3.4 系统架构



3.5 核心功能

3.5.1 数据导入

```
# API 方式
curl -X POST http://localhost:5001/api/import \
  -H "Content-Type: application/json" \
  -d '{"documents": [{"content": "张三是李四的父亲，李四在谷歌公司工作。"}]}'

# 从文件导入
curl -X POST http://localhost:5001/api/import-file \
```

```
-H "Content-Type: application/json" \  
-d '{"file_path": "data/sample_documents.txt"}'
```

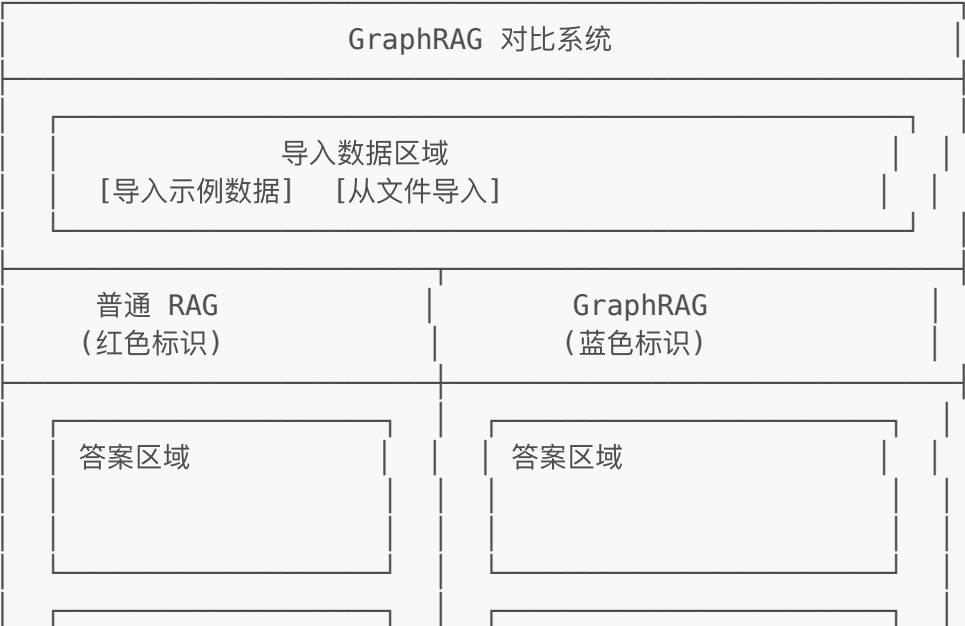
3.5.2 对比查询

```
curl -X POST http://localhost:5001/api/compare-query \  
-H "Content-Type: application/json" \  
-d '{"query": "张三和谷歌公司有什么关系?"}'
```

3.5.3 API 端点

端点	方法	功能
/api/import	POST	导入文档到知识图谱
/api/import-file	POST	从文件导入到知识图谱
/api/query	POST	GraphRAG 查询
/api/graph	GET	获取图谱结构
/api/status	GET	服务状态
/api/rag-import	POST	导入文档到 RAG
/api/rag-query	POST	普通 RAG 查询
/api/rag-import-file	POST	从文件导入到 RAG
/api/compare-query	POST	对比查询

3.6 前端界面





3.7 输出示例

问题：张三和谷歌公司有什么关系？

系统	答案
普通 RAG	根据文档"张三是李四的父亲，李四在谷歌公司工作"，可以推断张三与谷歌公司有间接关系。
GraphRAG	张三通过他的父亲李四与谷歌公司产生联系。路径：张三 → 父亲 → 李四 → 工作单位 → 谷歌公司

3.8 适用场景

- 教学演示对比
- 产品功能展示
- 用户交互界面
- 实时查询系统
- 需要较多资源运行

四、对比总结

4.1 功能对比

特性	内存版	Neo4j版	前后端对比系统
存储后端	NetworkX (内存)	Neo4j (持久化)	Neo4j + 向量索引
数据持久化	否	是	是
Web 界面	无	无	有
图谱可视化	无	Neo4j Browser	前端展示

特性	内存版	Neo4j版	前后端对比系统
RAG 对比	✗ 无	✗ 无	✓ 有
复杂度	★ 简单	★★ 中等	★★★ 复杂
资源需求	低	中	高

4.2 选择指南

场景	推荐版本
快速原型/教学演示	内存版
生产环境/数据持久化	Neo4j 版
功能展示/用户界面	前后端对比系统
理解 GraphRAG 核心原理	内存版
理解图数据库存储	Neo4j 版
对比 GraphRAG 与 RAG	前后端对比系统

4.3 学习路径建议

1. 第一步：运行内存版 `local_graphrag_ollama.py`，理解 GraphRAG 核心流程

2. 第二步：运行 Neo4j 版 `local_graphrag_neo4j.py`，理解持久化存储

3. 第三步：启动前后端对比系统，体验完整功能

五、快速开始

5.1 环境准备

```
# 1. 克隆/下载项目

# 2. 安装 Python 依赖
pip install -r requirements.txt

# 3. 确保 Ollama 运行
ollama serve
ollama pull qwen2.5
ollama pull mxbai-embed-large

# 4. (可选) 启动 Neo4j
neo4j start
```

5.2 一键运行

```
# 演示一：内存版
python local_graphrag_ollama.py

# 演示二：Neo4j 版
./start_graphrag.sh

# 演示三：前后端对比系统
./start_full.sh
```

文档版本: v1.0 生成日期: 2026年5月